Lab-3_ICA-and-PCA-for-brain-networks

October 21, 2026

[21]:
```
# !pip install netneurotools

# !pip install brainspace nilearn nibabel brainstat
```

[22]:
```
from nilearn import datasets, image
import nilearn.plotting as niplot
from nilearn.decomposition import CanICA
from brainspace.datasets import load_group_fc, load_parcellation, load_conte69
from brainspace.gradient import GradientMaps
from brainspace.utils.parcellation import map_to_labels
from brainspace.plotting import plot_hemispheres
from brainstat.datasets import fetch_yeo_networks_metadata
from brainstat.datasets import fetch_parcellation
from brainstat.datasets import fetch_mask, fetch_template_surface
import matplotlib as plt
import numpy as np
from brainspace.mesh.mesh_io import read_surface
import os
import pandas as pd
import matplotlib.pyplot as plt
import nibabel as nb
from IPython.display import Image
from nilearn.plotting import plot_prob_atlas
from nilearn.image import iter_img
from nilearn.plotting import plot_stat_map, show
```

1 Lab 3 - ICA and PCA for detecting brain networks

In this lab, we are going to explore the brain dynamics via more "traditional" approaches such as independent component analysis (ICA) and principal component analysis (PCA).

Both approaches are useful to identify the regions that show similar temporal dynamics, in other words, regions that correlate with each other.

More technically, the earlier approach is looking for seperate components that maximize the explained variance in the given signal, whereas the latter approach is looking for one common component that explains the most variance.

2 ICA

With ICA, we aim to detect the regions that show correlated functional dynamics with eachother. To start with, we are going to download a series of functional images from Nilearn's library. Subjects children changing between 3 and 12 years and are recorded while they are watching a pixar movie (Published here and available here. The tutorial is adapted from here).

[23]:
```
# First, change to working directiory to Lab folder
# os.chdir('path/to/Lab-3_ICA-and-PCA-for-brain-networks/')
import os

os.chdir("./")
```

[24]:
```
subjects = 15  # How many subjects' data we want to download

rest_dataset = datasets.fetch_development_fmri(n_subjects=subjects)
func_filenames = rest_dataset.func  # list of 4D nifti files for each subject

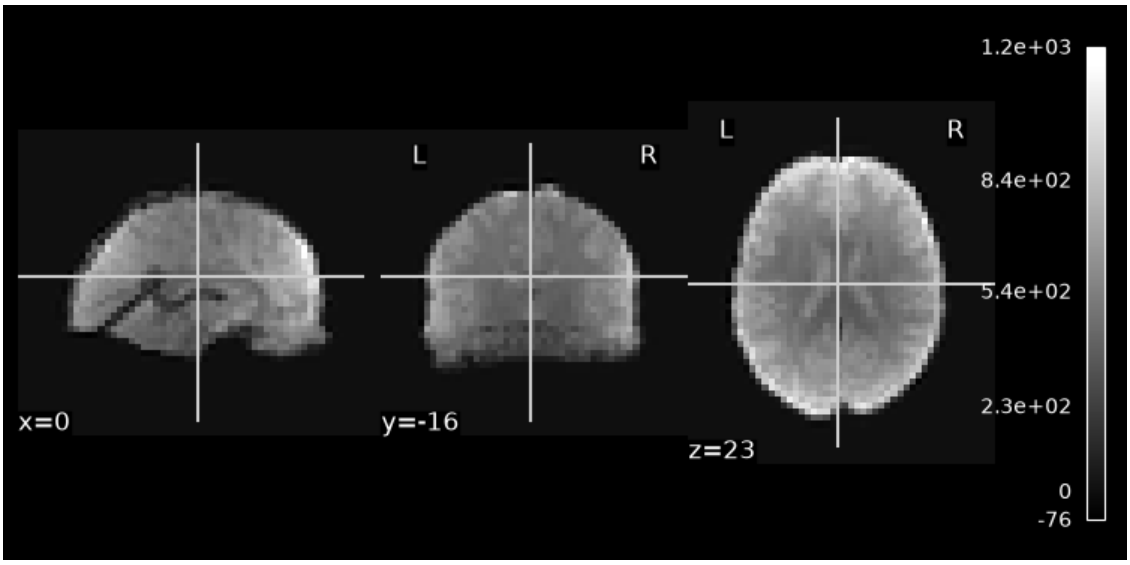
# print basic information on the dataset
print(f"First functional nifti image (4D) is at: {rest_dataset.func[0]}")
mean = image.mean_img(rest_dataset.func[0])
```

```
[fetch_development_fmri] Dataset directory found:
/home/pawlowiczf/nilearn_data/development_fmri
First functional nifti image (4D) is at:
/home/pawlowiczf/nilearn_data/development_fmri/sub-pixar123_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz
```

Let's have a look at a mean image of one subject. Each point on the brain has a time-series with 168 time points. So, each image is a set of time series showing the functional dynamics of that specific point of the brain.

[25]:
```
niplot.plot_epi(mean)
print(image.load_img(rest_dataset.func[0]).shape)
```

```
(50, 59, 50, 168)
```



3 Conducting the ICA

The detailed method is explained here. In short, this methods create spatial independent component maps for each subject, and then creates group-level maps based on the spatial consistency of those maps. A snapshot of the workflow can be seen below.

[26]:
```
Image(filename="ica_workflow.png")
```

[26]:

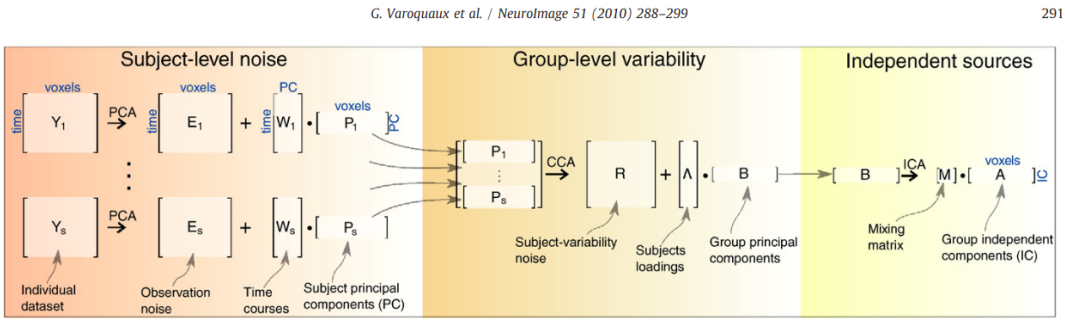


Fig. 1. Diagram of the estimation procedure: successive estimation steps are applied from left to right, starting from individual datasets, to end with group-level independent components.

[27]:
```
# Let's run the model on our data

canica = CanICA(
    n_components=20,
    memory="nilearn_cache",
    memory_level=2,
```

```
    verbose=10,
    mask_strategy="whole-brain-template",
    random_state=0,
    standardize="zscore_sample",
)
canica.fit(func_filenames)

# Retrieve the independent components in brain space. Directly
# accessible through attribute `components_img_`.
canica_components_img = canica.components_img_
# components_img is a Nifti Image object, and can be saved to a file with
# the following line:
canica_components_img.to_filename("canica_resting_state.nii.gz")

# Save the component maps seperately
canica_components_img = canica.components_img_
```

```
[CanICA.fit] Loading data from [
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar123_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar124_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar125_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar001_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar002_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar003_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar004_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar005_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar006_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar007_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar008_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar009_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar010_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar011_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar012_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
```

```
]
[CanICA.fit] Loading data from [
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar123_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar124_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar125_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar001_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar002_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar003_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar004_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar005_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar006_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar007_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar008_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar009_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar010_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar011_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
 /home/pawlowiczf/nilearn_data/development_fmri/sub-pixar012_task-pixar_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz,
]
[CanICA.fit] Computing mask
[CanICA.fit] Resampling mask
[CanICA.fit] Finished fit
[CanICA.fit] Loading data
[Memory]0.1s, 0.0min    : Loading randomized_svd from nilearn_cache/joblib/sklea
rn/utils/extmath/randomized_svd/650dd1e05c0aa4be2b6b4dc83ea0bbe8
_____________________________________randomized_svd cache loaded - 0.0s, 0.0min
[CanICA.fit] Computing image from signals
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/ebb15c4501a3937d5b825b3983d0651f
________________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/e05454e44aa3a2f9fd5fa0b154a63f34
________________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
```

```
mposition/_fastica/fastica/bf1a95389146c67deb7c8d8a28aef35d
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/9e52732e0897e3df5396144ed1f4288b
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/e2f754604295790d75ff67f4f1be94f2
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/2354f3bdc76282a5b70f7c9ede2a3040
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/a923526db7d2ff49618623690a995257
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.2s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/60d32185cf472d667f082b5fbe29522f
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.3s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/a28d88a90ff788526afc232645330247
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[Memory]0.3s, 0.0min    : Loading fastica from nilearn_cache/joblib/sklearn/deco
mposition/_fastica/fastica/fb0a74a751102d248a59c1ee37544b19
_____________________________________________fastica cache loaded - 0.0s, 0.0min
[CanICA.fit] Computing image from signals

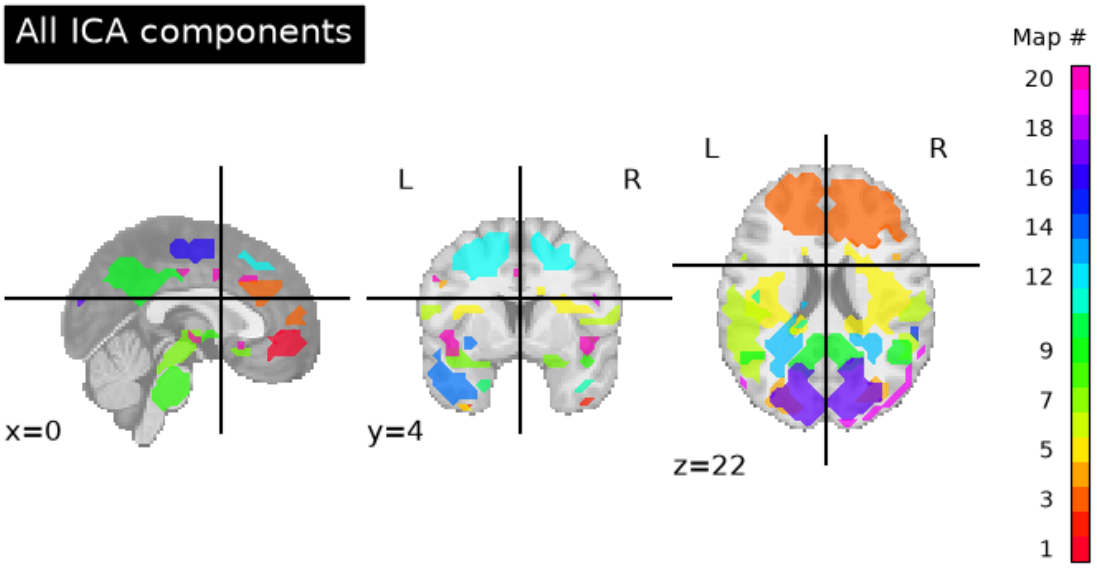
[Parallel(n_jobs=1)]: Done   1 tasks      | elapsed:    0.0s
[Parallel(n_jobs=1)]: Done   4 tasks      | elapsed:    0.0s
[Parallel(n_jobs=1)]: Done   7 tasks      | elapsed:    0.1s
[Parallel(n_jobs=1)]: Done  10 out of  10 | elapsed:    0.1s finished
```

Plot all ICA components together. All different colors show the regions have similar temporal dynamics These maps are also known as "networks", so we identified the "brain networks during movie-watching"!

[28]:
```
plot_prob_atlas(canica_components_img, title="All ICA components",␣
↪draw_cross=True)
```

```
/home/pawlowiczf/Machine-Learning-applied-to-Neuroimaging-and-
Neuroscience/.venv/lib/python3.12/site-packages/numpy/ma/core.py:2903:
UserWarning: Warning: converting a masked element to nan.
  _data = np.array(data, dtype=dtype, copy=copy,
```

[28]:
```
<nilearn.plotting.displays._slicers.OrthoSlicer at 0x750ad8f8f050>
```



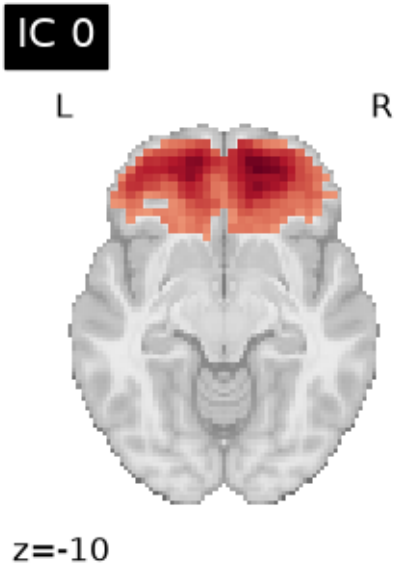
[29]:
```
# 70% of the variance is explained by these networks. This is good!
# The rest can be missing due to factors such as small sample size,
# inconsistincies in the sample, or noise in the data

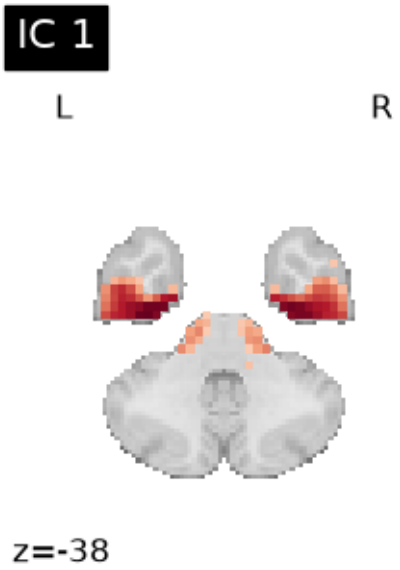
print(
    "Total explained variance of the brain activity by the identified networks:␣
 ↪",
    canica.score(canica.components_img_),
)
```

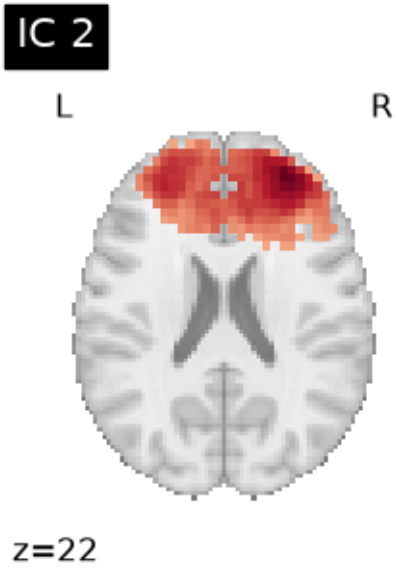
```
[Memory]1.9s, 0.0min    : Loading _explained_variance from nilearn_cache/joblib/
nilearn/decomposition/_base/_explained_variance/f125b3199867fafb3beea1ba2a876474
_____________________________________explained_variance cache loaded - 0.0s, 0.0min
Total explained variance of the brain activity by the identified networks:
0.6998473061191766
```

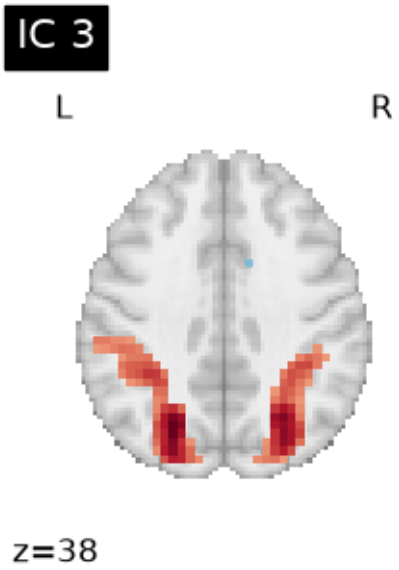
[30]:
```
# Let's check them one by one

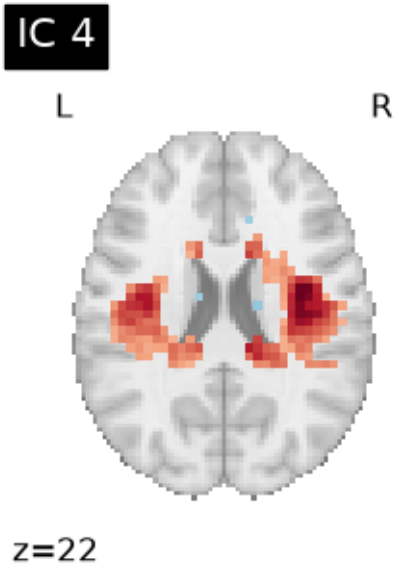
for i, cur_img in enumerate(iter_img(canica_components_img)):
    plot_stat_map(
        cur_img,
        display_mode="z",
        title=f"IC {int(i)}",
        cut_coords=1,
        colorbar=False,
    )
```

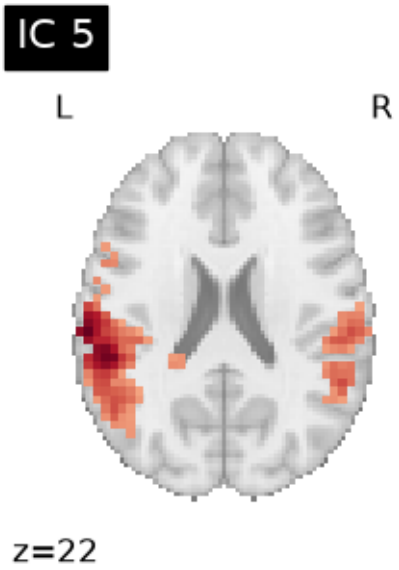

IC 0
L
R
z=-10

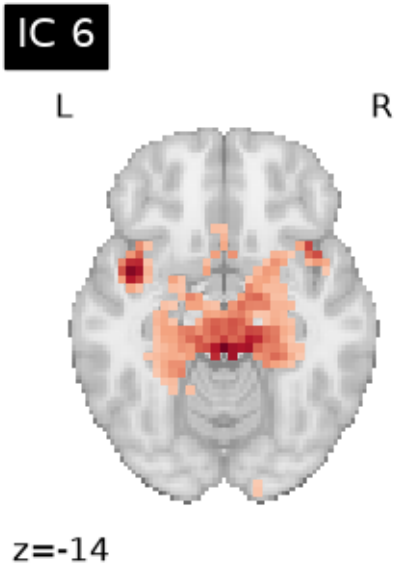

IC 1
L
R
z=-38

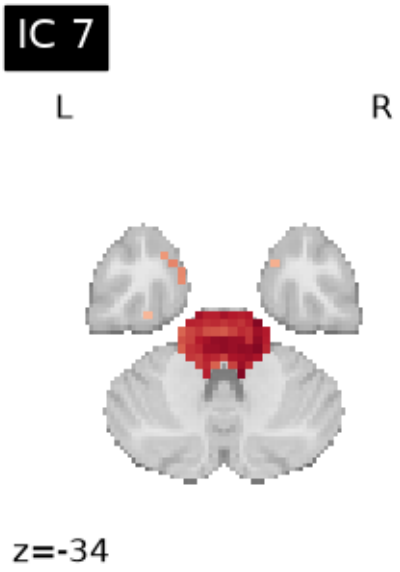

IC 2
L
R
z=22

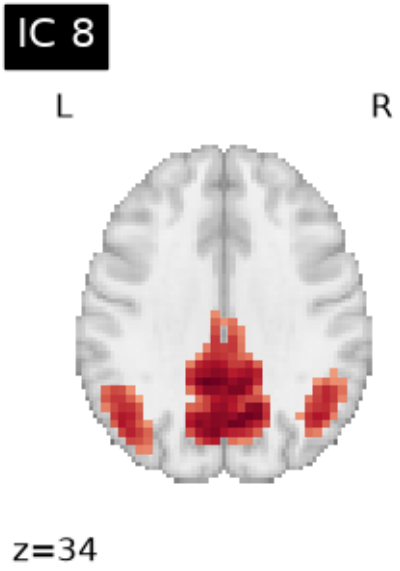

IC 3
L
R
z=38

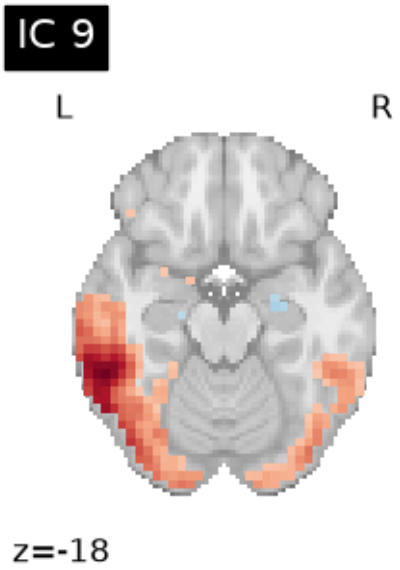

IC 4
L
R
z=22

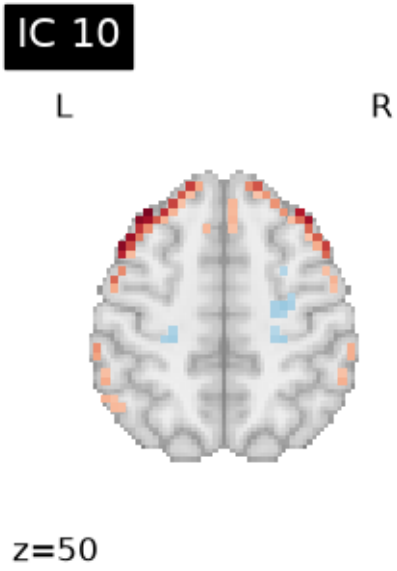

IC 5
L
R
z=22

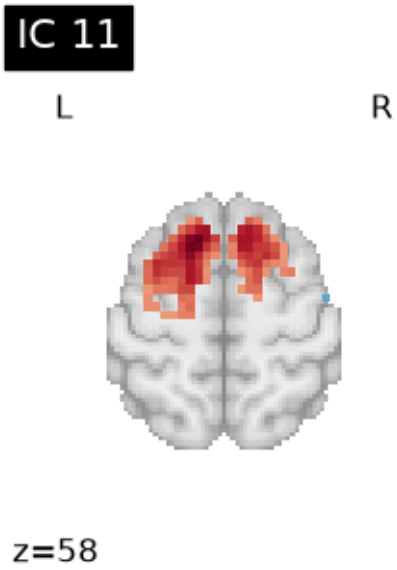

IC 6
L
R
z=-14

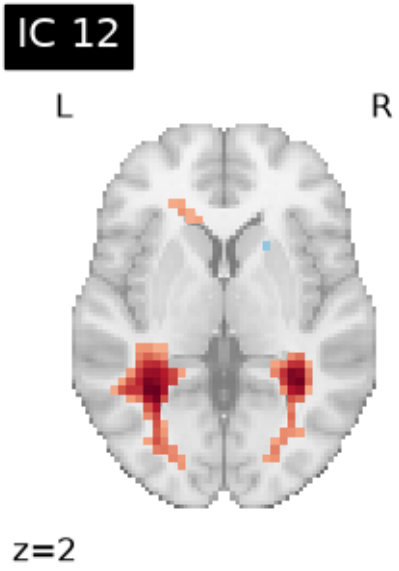

IC 7
L
R
z=-34

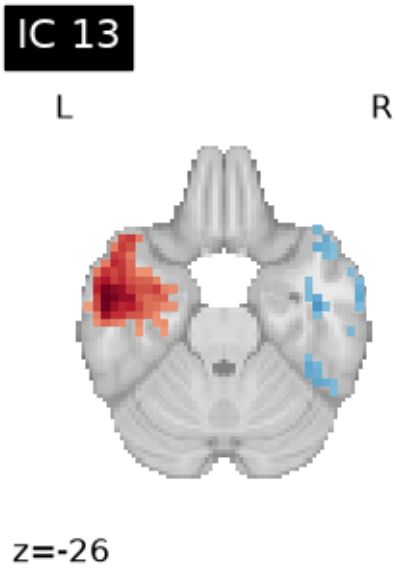

IC 8
L
R
z=34

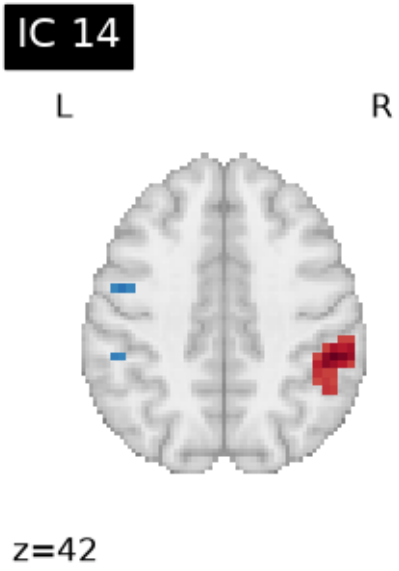

IC 9
L
R
z=-18

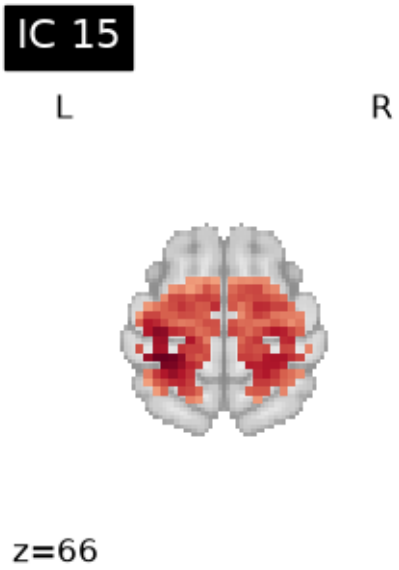

IC 10
L
R
z=50

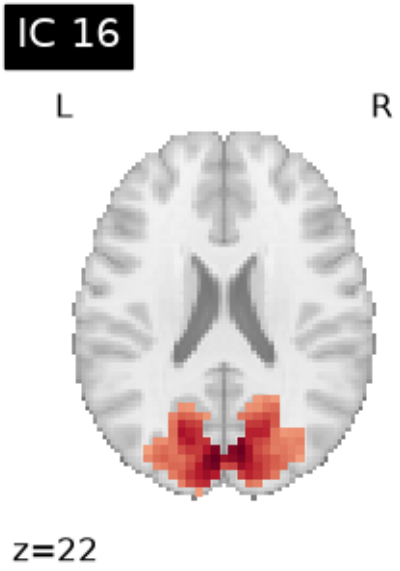

IC 11
L
R
z=58

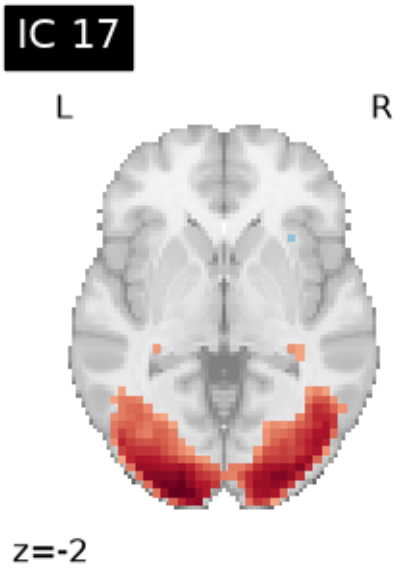

IC 12
L
R
z=2

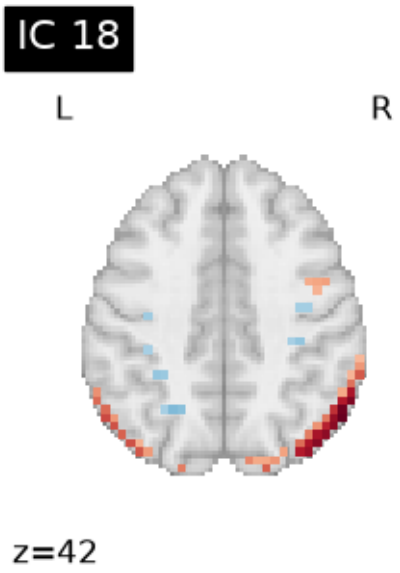

IC 13
L
R
z=-26

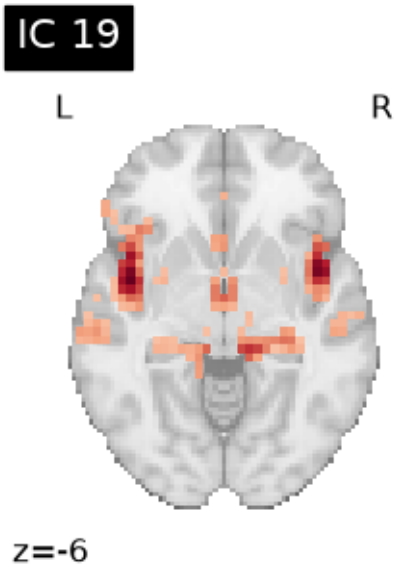

IC 14
L
R
z=42


IC 15
L
R
z=66


IC 16
L
R
z=22


IC 17
L
R
z=-2





3.0.1 Okay, they look nice, but what do these maps show?

Now we detected the networks, but we do not know what function or feature they are associated with. Luckily, there is a tool that excels at "decoding" brain maps into their associated structure or functions, called NeuroSynth. It basically utilizes the association between the brain coordinates

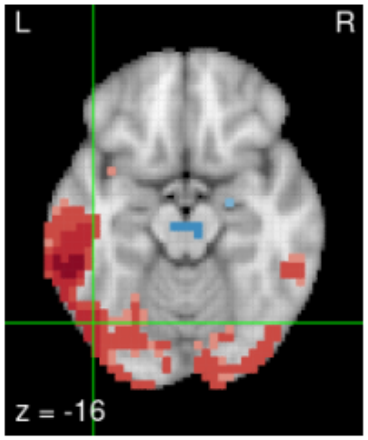
and cognitive functions reported in (a lot of) previous studes, so it is a kind of meta-search. The tool is capable of doing much more, go ahead and play with it!

In our specific case, we upload those network maps into a NeuroVault repository, and then use NeuroSynth's decode function there:

P.s.: There is a nicer way to do the same via python code using neurosynth api and create beatiful wordclouds, but for some reason it tells me "it is deprecated".

[31]:
```
Image(filename="ica.png")
```

[31]:



Intensity: 0.03

What's here?

X: -46 Y: -70 Z: -16

Layers

ica0

anatomical

Color palette:

intense red-blu

Positive/Negative:

both

Crosshairs

Pan/zoom

Labels

Thresholds:

0 0

Opacity:

1

Term similarity

To compare the decoded image against a Neurosynth term, click on an arrow below.

Show 10 entries

Search:

analysis	corr
inferior temporal	0.298
word form	0.253
visual word	0.251
fusiform	0.247
fusiform gyrus	0.224
fusiform face	0.211
face ffa	0.198
ffa	0.196
face	0.196
fusiform gyri	0.191

Showing 1 to 10 of 1,307 entries

3.0.2 Now I know what my IC-generated networks mean. How is that useful?

ICA is a common method to detect brain networks and it has high reproducibility. The consistency of the similar networks were shown many times in the literature. Therefore, it is possible to the check the spatial distribution of the specific networks are different in special populations (e.g. visual network in blind people,), or in specific conditions (e.g. attention network during resting vs. during movie watching).

4 PCA

In order to apply PCA on our data, we are going to follow the method in Marguiles's paper. This method allows decreasing the dimension of the brain signal into a very small number (1-3), and summarize the brain function-structure with nice visualization. However, instead of applying PCA on the data directly (like ICA), this method defines the functionally similar regions first. This helps us to see the regions that are close to each other:

[32]:
```
Image("pca.png")
```

[32]:

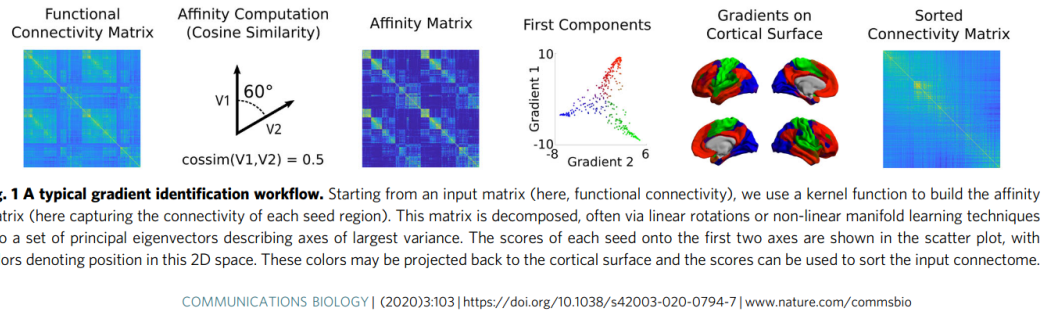


Fig. 1 A typical gradient identification workflow. Starting from an input matrix (here, functional connectivity), we use a kernel function to build the affinity matrix (here capturing the connectivity of each seed region). This matrix is decomposed, often via linear rotations or non-linear manifold learning techniques into a set of principal eigenvectors describing axes of largest variance. The scores of each seed onto the first two axes are shown in the scatter plot, with colors denoting position in this 2D space. These colors may be projected back to the cortical surface and the scores can be used to sort the input connectome.

The following steps are a combination of the BrainSpace (relevant paper, more methods-rich), BrainStat, and Micapipe toolboxes. The procedure described in the Marguiles's paper is applied via BrainSpace, and the other two toolboxes are mostly used for visualization. This method is mostly applied on the "surface space" rather than "voxel space", so the existing toolboxes adopted this tradition. This is why the visuals will be a bit different compared to ICA,

[33]:
```
# Prepare the surface meshes. This code block is for preaparing the upcoming␣
↪figures

labeling_conte = load_parcellation("schaefer", scale=400, join=True)
c69_lh, c69_rh = load_conte69()


# Load fsaverage5
fs5_lh = read_surface("lh.pial", itype="fs")
```

```
fs5_rh = read_surface("rh.pial", itype="fs")


# Load annotation file in fsaverage5
annot_lh_fs5 = nb.freesurfer.read_annot(
    "lh.Schaefer2018_400Parcels_7Networks_order.annot"
)
annot_rh_fs5 = (
    nb.freesurfer.read_annot("rh.Schaefer2018_400Parcels_7Networks_order.
 ↪annot")[0]
    + 200
)

# replace with 0 the medial wall of the right labels
annot_rh_fs5 = np.where(annot_rh_fs5 == 200, 0, annot_rh_fs5)

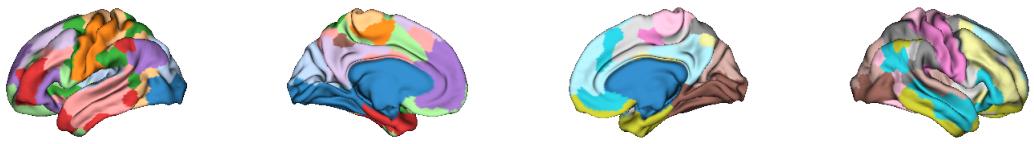
# fsaverage5 labels
labels_fs5 = np.concatenate((annot_lh_fs5[0], annot_rh_fs5), axis=0)
```

The data we are going to use is a set of resting-state data from HCP dataset, available in BrainSpace's library. It contains the brain activity during when the subject do not do anything particular, hence measuring natural dynamics of the brain. However, before PCA is applied, the data is already reduced to 400 hundred regions by using the parcellation below. Mean time series within the voxels that corresponding the same regions were extracted, and averaged across subjects:

[34]:
```
yeo_networks = fetch_parcellation("fsaverage5", "yeo", 7)
network_names, yeo_colormap = fetch_yeo_networks_metadata(7)

plot_hemispheres(
    c69_lh,
    c69_rh,
    labeling_conte,
    embed_nb=True,
    cmap="tab20",
    nan_color=(0.7, 0.7, 0.7, 1),
    size=(1400, 200),
    zoom=1.45,
)
```

[34]:

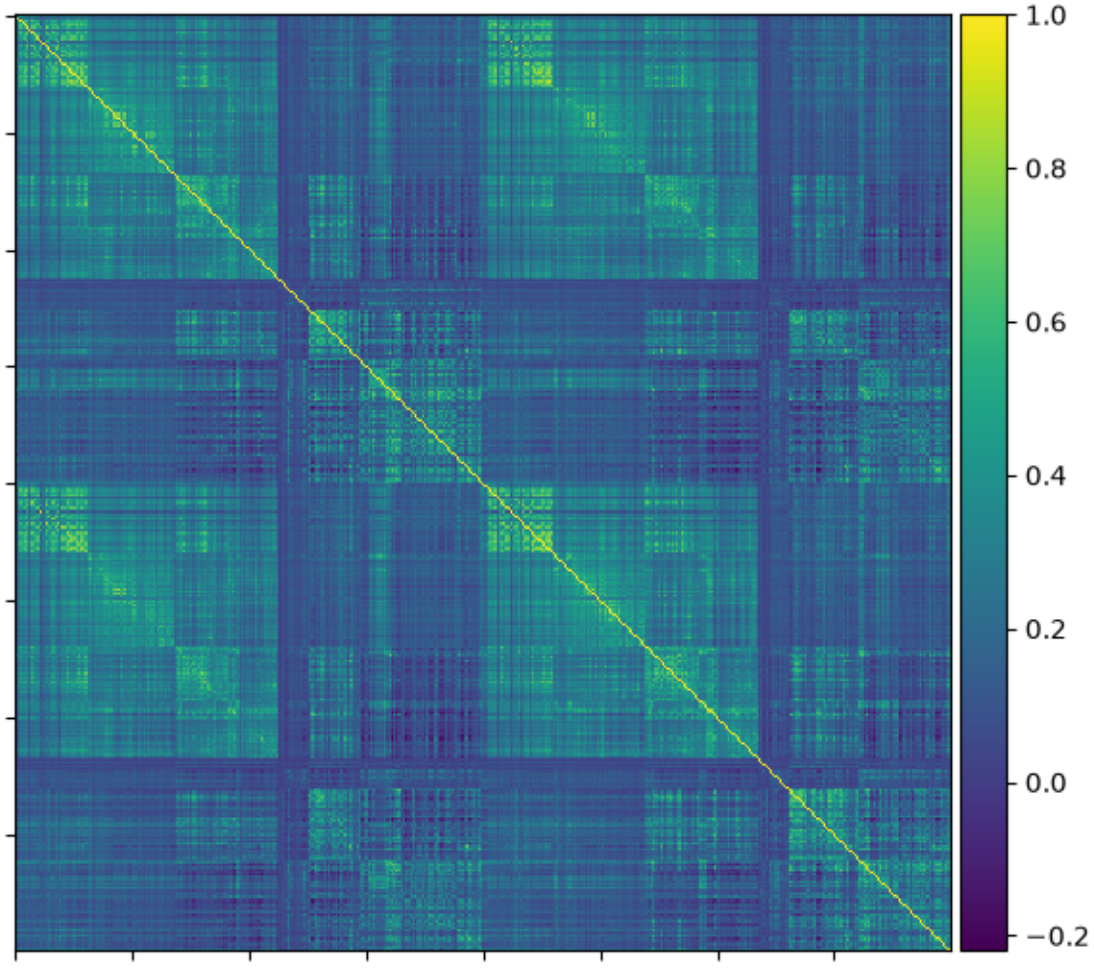


Then, each time series were correlated with each other, creating a 400x400 correlation matrix. As can be seen, some regions have high correlation and already form some clusters that is visible to

naked eye:

[35]:
```
conn_matrix = load_group_fc("schaefer", scale=400)
niplot.plot_matrix(conn_matrix, cmap="viridis")
```

[35]:
```
<matplotlib.image.AxesImage at 0x750b08f1f800>
```



With the following simple line of code, we are going to calculate the affinity matrix and apply PCA on the thresholded affinity matrix. This will return the 10 principal components, and their eigenvalues (explained variances).

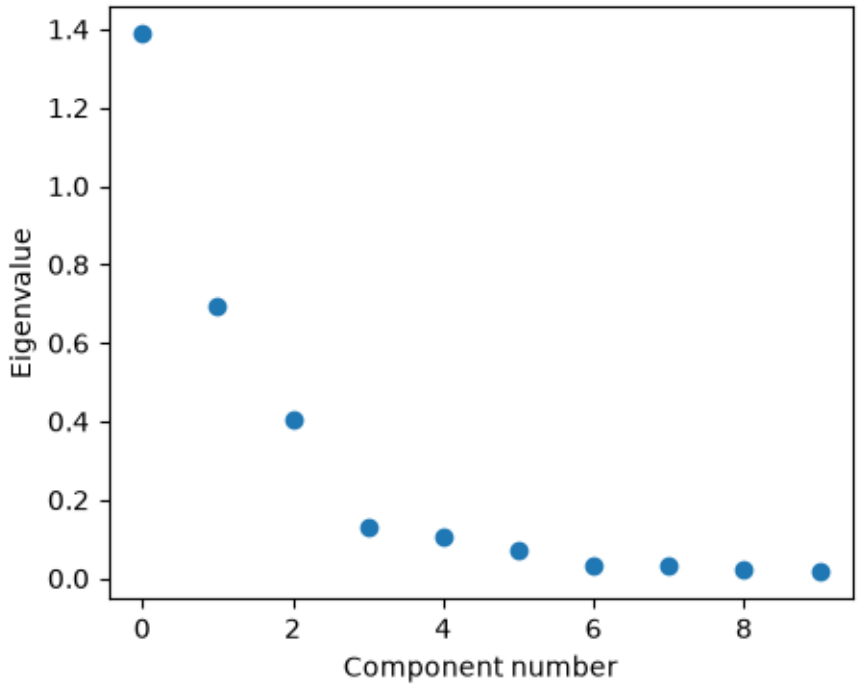
[36]:
```
gm = GradientMaps(n_components=10, approach="pca")
gm.fit(conn_matrix, sparsity=0.8)
```

[36]:
```
GradientMaps(approach='pca')
```

In the plot below, you can see the eigenvalues of the components. The first 3 principal components have the highest eigenvalues, hence they explain the most variances in the affinity matrices.

[37]:
```
fig, ax = plt.subplots(1, figsize=(5, 4))
ax.scatter(range(gm.lambdas_.size), gm.lambdas_)
ax.set_xlabel("Component number")
ax.set_ylabel("Eigenvalue")

plt.show()
```



Let's look the distribution of the principal component values over regions

[38]:
```
# Mask of the medial wall on fsaverage 5
mask_fs5 = labels_fs5 != 0

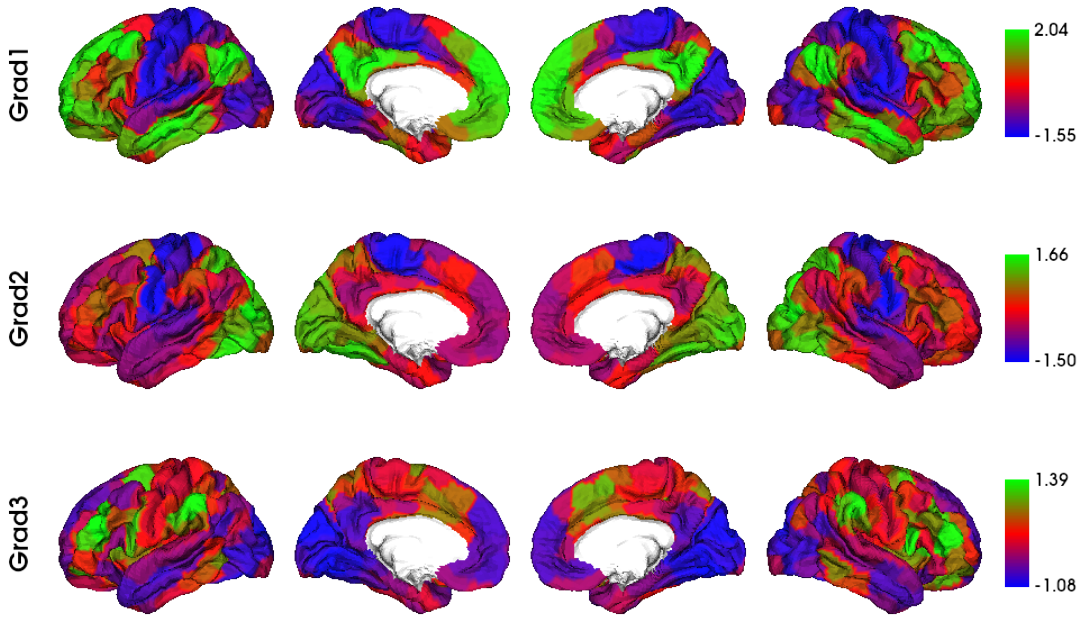
# Map gradients to original parcels
grad = [None] * 3

for i, g in enumerate(gm.gradients_.T[0:3, :]):
    grad[i] = map_to_labels(g, labels_fs5, fill=np.nan, mask=mask_fs5)

# Plot Gradients RdYlBu
plot_hemispheres(
    fs5_lh,
    fs5_rh,
    array_name=grad,
    size=(1000, 600),
```

```
    cmap="brg",
    embed_nb=True,
    label_text={"left": ["Grad1", "Grad2", "Grad3"]},
    color_bar="right",
    zoom=1.25,
    nan_color=(1, 1, 1, 1),
)
```

[38]:



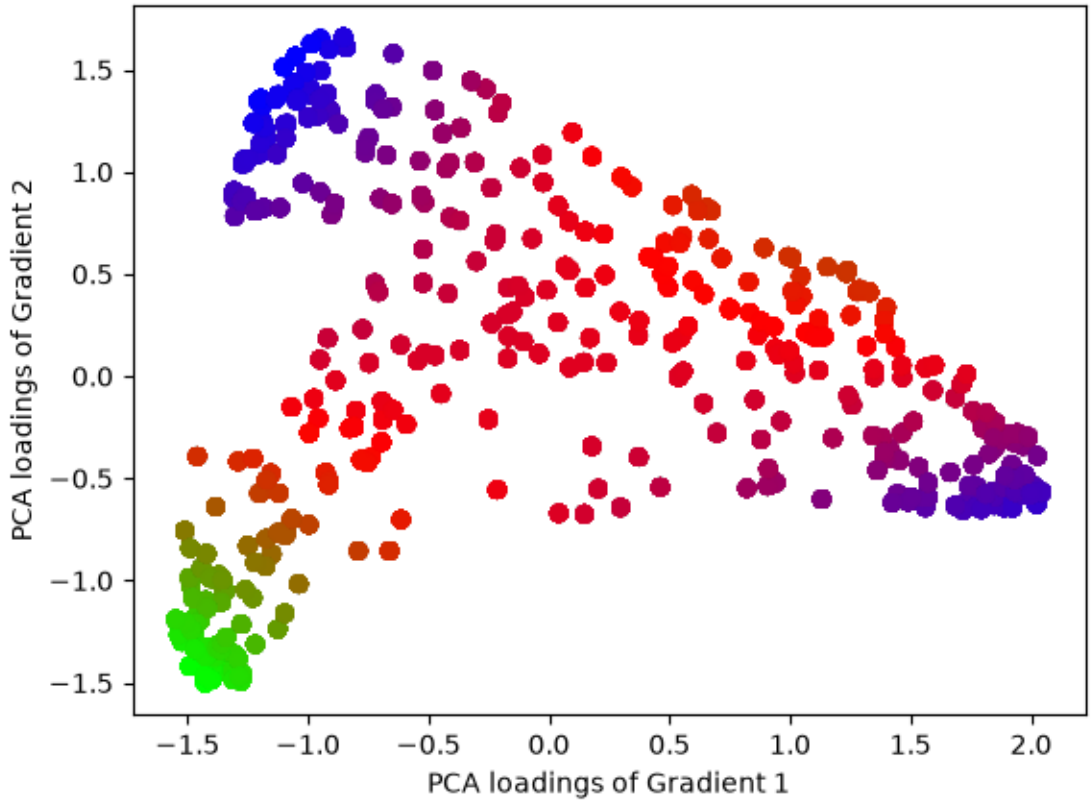
4.0.1 Cool. But what am I looking at?

Let's focus on the first principal component, "Grad1". If you look at the colorbar, you will see that the lowest values are coded with blue, and the highest values are coded with green. The interpretation of this method is that brain function is organized in this "color-axis". Regions that are on the extreme of this distribution are functionally similar to each other, and most of the brain function can be explained by the commonalities of these regions. You can see the distribution of the regions in the 2D space in the following figure:

[39]:

```
plt.scatter(
    np.array(grad)[0, :],
    np.array(grad)[1, :],
    c=(np.array(grad)[0, :] * np.array(grad)[1, :]),
    cmap="brg",
)
plt.xlabel("PCA loadings of Gradient 1")
plt.ylabel("PCA loadings of Gradient 2")
```

[39]: Text(0, 0.5, 'PCA loadings of Gradient 2')



4.0.2 What do these "similar regions" correspond to?

We will apply a decoding approach similar to the ICA maps. First, we are going to retrieve a an atlas of 7 functional connectivity network (previously defined by ICA!), and check which which PC loadings correspond to which networks.

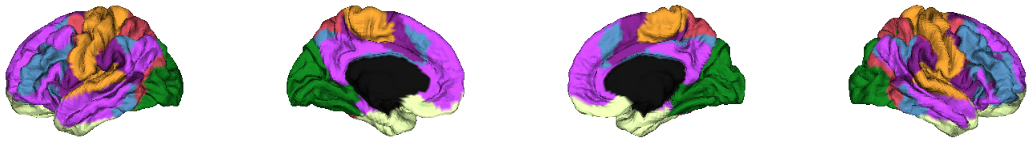
The network atlas is displayed below:

[40]:
```
surfaces = fetch_template_surface("fsaverage5", join=False)
yeo_networks = fetch_parcellation("fsaverage5", "yeo", 7)
network_names, yeo_colormap = fetch_yeo_networks_metadata(7)

plot_hemispheres(
    surfaces[0],
    surfaces[1],
    yeo_networks,
    embed_nb=True,
    cmap="yeo7",
    nan_color=(0.7, 0.7, 0.7, 1),
    size=(1400, 200),
```

```
    zoom=1.45,
)
```

[40]:



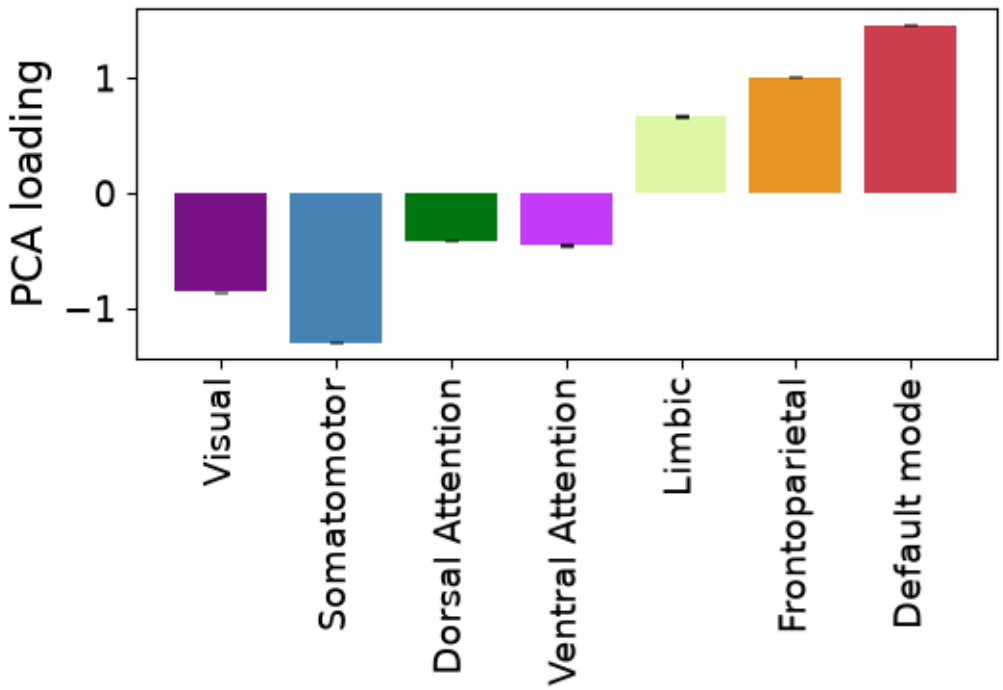
[41]:

```
import matplotlib.pyplot as plt
from scipy.stats import sem

from brainstat.context.resting import yeo_networks_associations

yeo_tstat_mean = yeo_networks_associations(grad[0], "fsaverage5")
yeo_tstat_sem = yeo_networks_associations(
    grad[0],
    "fsaverage5",
    reduction_operation=lambda x, y: sem(x, nan_policy="omit"),
)

plt.bar(
    np.arange(7),
    yeo_tstat_mean[:, 0],
    yerr=yeo_tstat_sem.flatten(),
    color=yeo_colormap,
    error_kw={"elinewidth": 5},
)
plt.xticks(np.arange(7), network_names, rotation=90, fontsize=14)
plt.yticks(fontsize=14)
plt.ylabel("PCA loading", fontdict={"fontsize": 16})
plt.gcf().subplots_adjust(left=0.2, bottom=0.5)
plt.show()
```



4.0.3 What does it mean?

In the above figure, you can see the quantification of the PC scores within the 7 networks we downloaded. As you can see, for the first PC, the visual and somatomotor networks have the highest concentration, and the default/frontoparietal networks have the lowest concentration. This implies that, brain function is mostly organized in the basic functions vs. higher functions axis. regions that are related to those basic cognitive functions are form a cluster, whereas the regions that are related to complex cognitive functions form another one.

Can you edit the code and check the function distribution of the second and third gradients?

4.0.4 Take-home message

Reltively simpler approaches such as ICA and PCA are still very suitable to examine the brain dynamics. They can lead to very complex analysis and are highly reproducible. Identify your needs depending on your hypothesis, and do not unnecessarily complicate your analysis pipeline.